# Object-Oriented Programming

Polymorphism

Ridwan Rismanto

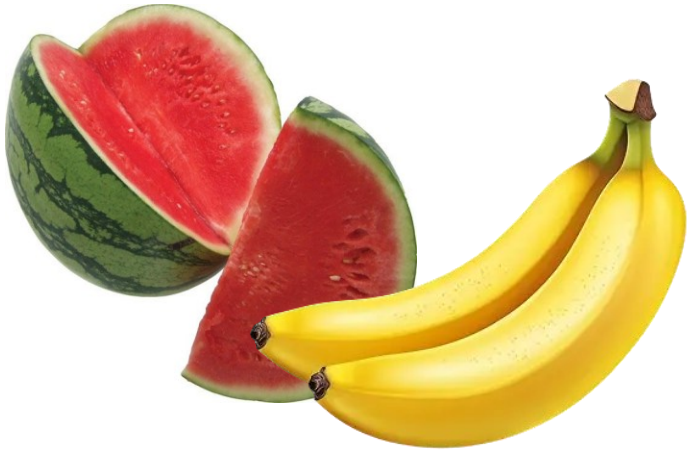
# Polymorphism

- Definition of Polymorphism
- Static Polymorphism
- Dynamic Polymorphism
- Polymorphic Arguments
- The instanceof keyword

# Definition

- It means 'can take various forms'
- Polymorphism in OOP is the ability to have several different class forms.
- Essentially, all classes in Java are polymorphic because, in addition to being of their own class type, they are also of the Object class type.
  - Since all classes in Java, including those we create ourselves, are derived from the Object class
- As an analogy, there is a class Animal and a class Cat, where Cat is a subclass of Animal. So we can say, all cats are animals,

# Examples of Polymorphism in the Real World



Humans can eat all types of fruit. But the way they eat them is different. Apples can be eaten, Bananas can also be eaten (since Apples and Bananas are in the Fruit category), only the way is different. Bananas need to be peeled first.



In this case, the Human object has a method EatFruit with a parameter Fruit object. This parameter can accept an object that is a subclass of Fruit (Apple, Banana, Watermelon, etc.). Then within the EatFruit method, it is detected whether the Fruit is an Apple, a Banana, or something else. Then it is treated differently (Apples are eaten directly, Bananas are peeled first, and so on).

# Types of Polymorphism

- There are two types of polymorphism:
- Static Polymorphism
  - That is when there is method overloading.
- Dynamic Polymorphism
  - That is when there is method overriding.

# Static Polymorphism

- Static Polymorphism in Java occurs if there is method overloading.
  - Method overloading: several methods with the same name but different signatures within a single class.
- At compile-time, Java knows which method is called by matching the signature of the called method.
- Therefore, static polymorphism is also referred to as compile-time polymorphism or static binding.

# Example

```java
public class DemoOverload
{
    // method 1
    public int add(int x, int y)
    {
        return x + y;
    }

    // method 2
    public int add(int x, int y, int z)
    {
        return x + y + z;
    }

    // method 3
    public int add(double x, int y)
    {
        return (int)x + y;
    }

    // method 4
    public int add(int x, double y)
    {
        return x + (int)y;
    }
}
```

```java
public class TestOverload
{
    public static void main(String[] args)
    {
        DemoOverload demo = new DemoOverload();

        System.out.println(demo.add(2,3));      //method 1 called

        System.out.println(demo.add(2,3,4));    //method 2 called

        System.out.println(demo.add(2,3.4));    //method 4 called

        System.out.println(demo.add(2.5,3));    //method 3 called
    }
}
```

# Example

- In the example above, there are four types of the add() method.
- The Java compiler will look when these methods are called and match their signatures to determine which method will be executed.

# Dynamic Polymorphism

- Dynamic Polymorphism occurs when an object of the parent class type is created but the subclass constructor is called, and there is an overriding method that is invoked.
  - Creating an object of the parent class type by calling the subclass constructor:

```
Employee emp = new Manager();
```

    - Where the Manager class is a subclass of Employee.
    - In the example above, the object emp is of the Employee type but calls the Manager constructor.
    - It can be said that the emp object is of the Employee type but holds a reference to Manager.

# Example

```
public class Vehicle
{
    public void move()
    {
        System.out.println("Vehicles can move!!");
    }
}
```

```
public class MotorBike extends Vehicle
{
    public void move()
    {
        System.out.println("MotorBike can move and accelerate too!!");
    }
}
```

# Example (continued)

```
public class TestMotorBike
{
    public static void main(String[] args)
    {
        Vehicle vh = new MotorBike();

        vh.move();      // prints "MotorBike can move and accelerate too!!"

        vh = new Vehicle();

        vh.move();      // prints "Vehicles can move!!"
    }
}
```

```
D:\Java\Polimorfisme>java TestMotorBike
MotorBike can move and accelerate too!!
Vehicles can move!!
```

From the above experiment, we can see that, first, we create an object vh of the Vehicle type, but it holds a reference to the MotorBike class.

At the first invocation of the move() method, the method executed is the move() method of the MotorBike class.

Then the reference of object vh is returned to the Vehicle class.

After that, the executed move() method is that of Vehicle.

# Polymorphic Arguments

- These are method parameters that accept a value of the subclass type.

# Polymorphic Arguments

```
public class Pegawai
{
    // some class members
}

public class Manajer extends Pegawai
{
    // some class members
}

public class Test
{
    public static void Proses(Pegawai peg)
    {
        // do something
    }

    public static void main(String args[])
    {
        Manajer man = new Manajer();
        Proses(man);
    }
}
```

- The Manager class is a subclass of Employee.
- We can see in the Test class, there is a Process() method that accepts a parameter of the Employee type.
- However, when the Process method is called, its parameter is given an object man of the Manager type.

# The instanceof statement

- The instanceof statement is very useful for determining the original type of a polymorphic argument.
- Example of using instanceof:

```
if (peg instanceof Manajer)
{
    // lakukan tugas Manajer
}
```

- The above if condition will evaluate to true if the variable peg is of the Manager type.

# The instanceof statement

```java
public class Pegawai
{
    // some class members
}
```

```java
public class Manajer extends Pegawai
{
    // some class members
}
```

```java
public class Kurir extends Pegawai
{
    // some class members
}
```

```java
public class Test
{
    public static void Proses(Pegawai peg)
    {
        if (peg instanceof Manajer)
        {
            // lakukan tugas Manajer
        }
        else if (peg instanceof Kurir)
        {
            // lakukan tugas Kurir
        }
        else
        {
            // lakukan tugas lain
        }
    }

    public static void main(String args[])
    {
        Manajer man = new Manajer();
        Kurir kur = new Kurir();
        Proses(man);
        Proses(kur);
    }
}
```

# Casting Object

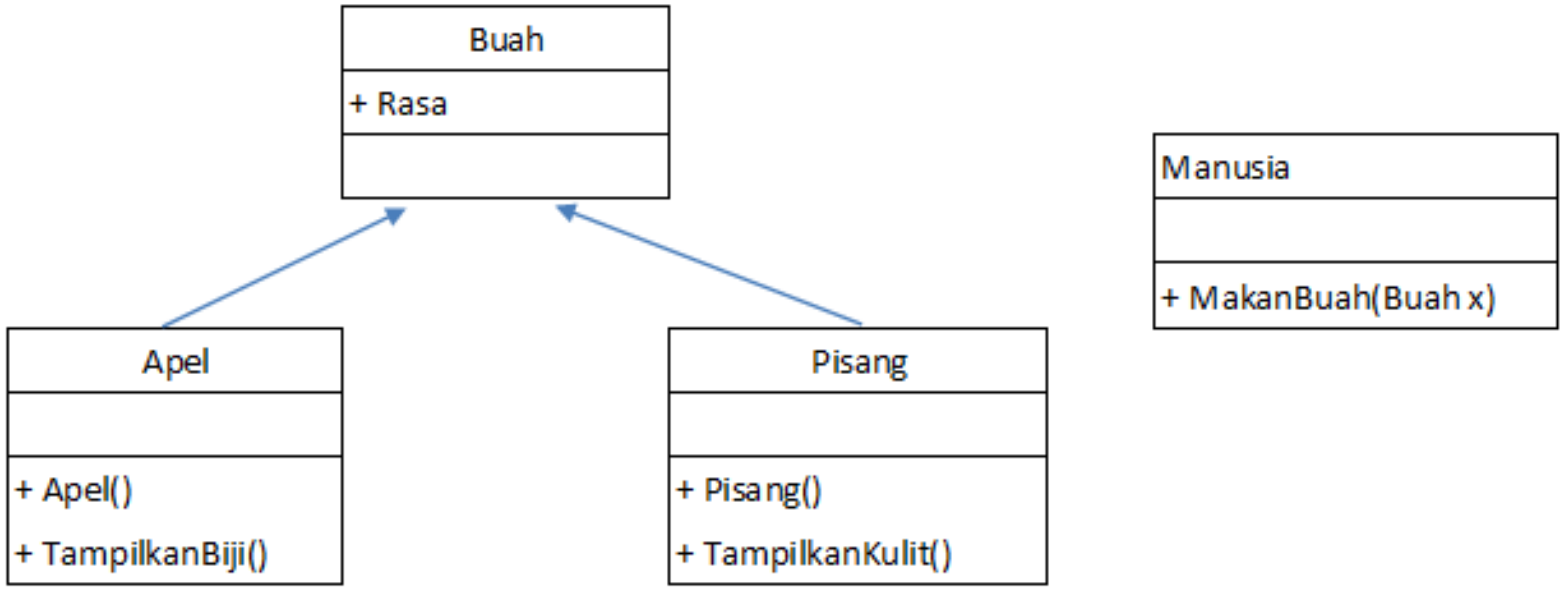
- The use of instanceof is often followed by casting an object (object conversion) from the parameter to its original type.
- For example, let's modify the Process method in the Test class above:

```
public static void Proses(Pegawai peg)
{
    if (peg instanceof Manajer)
    {
        Manajer man = (Manajer)peg;
        // lakukan tugas manajer
    }
    else if (peg instanceof Kurir)
    {
        Kurir kur = (Kurir)peg;
        // lakukan tugas Kurir
    }
    else
    {
        // lakukan tugas lain
    }
}
```

Casting the variable peg from Employee type to Manager

Casting the variable peg from Employee type to Courier

# Example of Polymorphism & Object Casting



# Example of Polymorphism & Object Casting (continued)

```
public class Buah
{
    public String Rasa;
}
```

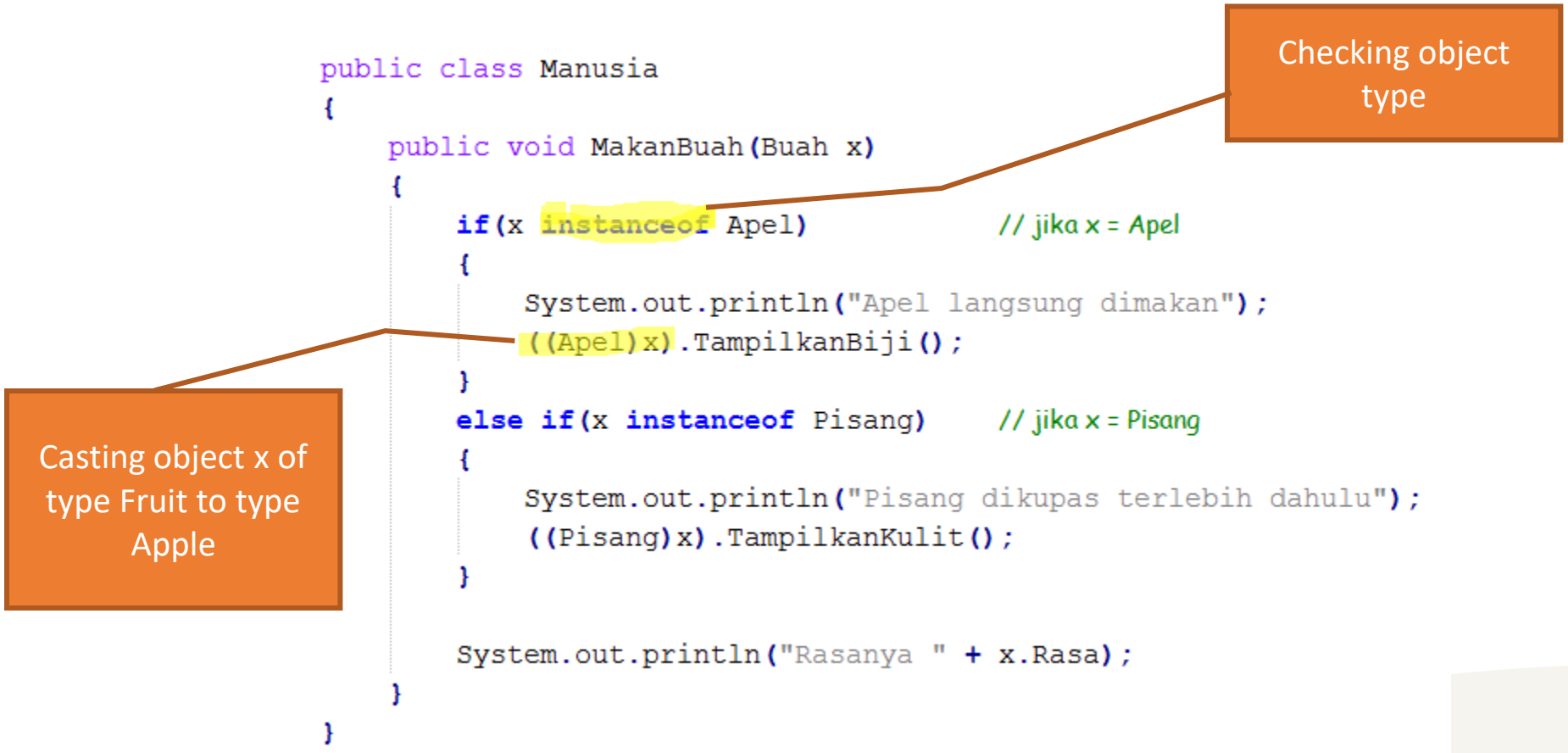
```
public class Apel extends Buah
{
    public Apel()
    {
        Rasa = "manis agak asem";
    }

    public void TampilkanBiji()
    {
        System.out.println("Biji apel berukuran kecil");
    }
}

public class Pisang extends Buah
{
    public Pisang()
    {
        Rasa = "sepet";
    }

    public void TampilkanKulit()
    {
        System.out.println("Kulit pisang berwarna kuning");
    }
}
```

# Example of Polymorphism & Object Casting (continued)




```
public class Manusia
{
    public void MakanBuah(Buah x)
    {
        if(x instanceof Apel)              // jika x = Apel
        {
            System.out.println("Apel langsung dimakan");
            ((Apel)x).TampilkanBiji();
        }
        else if(x instanceof Pisang)       // jika x = Pisang
        {
            System.out.println("Pisang dikupas terlebih dahulu");
            ((Pisang)x).TampilkanKulit();
        }

        System.out.println("Rasanya " + x.Rasa);
    }
}
```

# Example of Polymorphism & Object Casting (continued)

```
public class TestBuah
{
    public static void main(String[] b)
    {
        Manusia ilham = new Manusia();

        Apel myApel = new Apel();
        Pisang myPisang = new Pisang();

        ilham.MakanBuah(myPisang);
    }
}
```

The result:

```
D:\Java>javac TestBuah.java

D:\Java>java TestBuah
Pisang dikupas terlebih dahulu
Kulit pisang berwarna kuning
Rasanya sepet

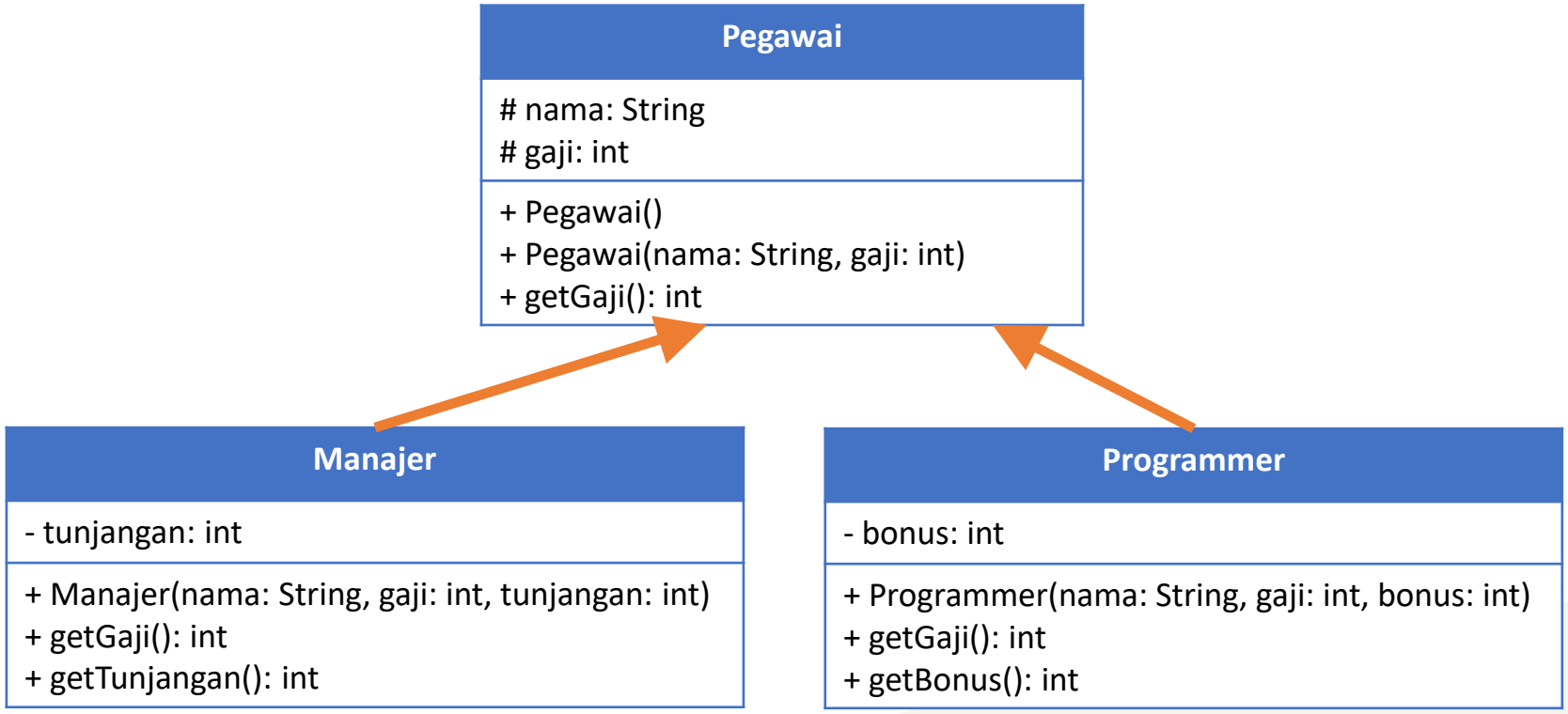
D:\Java>
```

# Exercise Example of Polymorphism & Object Casting

1. Add one more class, namely Durian, which is still a descendant of Fruit. Durian has a sweet and rich taste. The way to eat durian is to cut it open first. Adjust the MakanBuah() method from the Human class to be able to process eating durian.
2. Add the water content attribute to the Fruit class. Set the water content for each of the following classes through their constructors:
    - Banana: 100
    - Apple: 70
    - Durian: 80
    - Then display the water content on the screen, through the MakanBuah() method from the Human class.

# Exercise 1

- Create a program from the class diagram below:



# Exercise 1

- Then create the Bayaran and TestBayaran classes below for testing:

```
public class Bayaran
{
    public int hitungBayaran(Pegawai pg)
    {
        int uang = pg.getGaji();

        if(pg instanceof Manajer)
        {
            uang += ((Manajer)pg).getTunjangan();
        }
        else if(pg instanceof Manajer)
        {
            uang += ((Programmer)pg).getBonus();
        }

        return uang;

    }
}
```

```
public class TestBayaran
{
    public static void main(String[] args)
    {
        Manajer man = new Manajer("Agus", 800, 50)
        Programmer prog = new Programmer("Budi", 600, 30);
        Bayaran hr = new Bayaran();

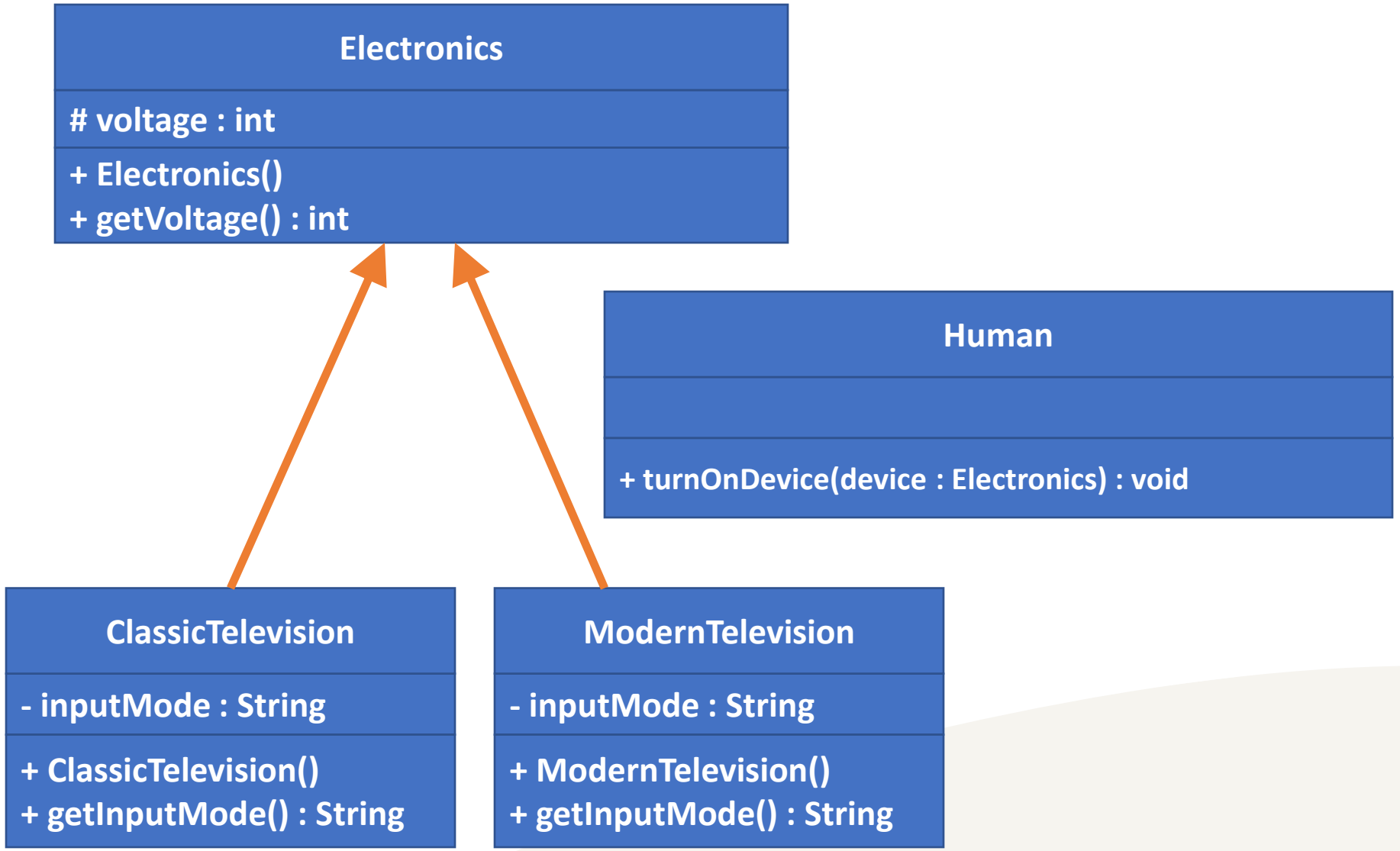
        System.out.println("Bayaran manajer: " + hr.hitungBayaran(man));
        System.out.println("Bayaran programmer: " + hr.hitungBayaran(prog));
    }
}
```

# Exercise 1

- The expected result:

```
Bayaran untuk Manajer : 850
Bayaran untuk Programmer : 630
```

# Exercise 2


Electronics
# voltage : int
+ Electronics()
+ getVoltage() : int
Human
+ turnOnDevice(device : Electronics) : void
ClassicTelevision
- inputMode : String
+ ClassicTelevision()
+ getInputMode() : String
ModernTelevision
- inputMode : String
+ ModernTelevision()
+ getInputMode() : String

# Exercise 2

- Create a program according to the class diagram above, then create the TestElectronics class as below:

```
public class TestElektronik
{
    public static void main(String[] args)
    {
        Manusia indro = new Manusia();
        TelevisiJadul tvjadul = new TelevisiJadul();
        TelevisiModern tvmodern = new TelevisiModern();

        indro.nyalakanPerangkat(tvjadul);
        indro.nyalakanPerangkat(tvmodern);
    }
}
```

The expected result:

```
D:\Java\Polimorfisme>java TestElektronik
Nyalakan televisi jadul dengan input: DVI
Voltase televisi: 220
Nyalakan televisi modern dengan input: HDMI
Voltase televisi: 220
```